

Cómo funciona el código del Torneo del Dilema del Prisionero

Guía para entender cada variable, arreglo y función, y por qué están ahí

Cómo usar esta guía (es interactiva):

- Todo lo que está subrayado con puntos es un **enlace**: al hacer clic saltas a la explicación de esa variable, función o sección. En el lector de PDF vuelve atrás con **Alt + ←**.
- Los recuadros **¿Por qué?** explican la razón de cada decisión de diseño.
- Los recuadros **Pruébalo** son preguntas para que traces el código tú mismo. Intenta responder antes de hacer clic en la respuesta.
- Los dibujos de **celdas** muestran cómo están los arreglos en memoria: arriba el valor, abajo el índice.

Contenido

1. El programa en 30 segundos
2. Las constantes: por qué 8 y 201 (y por qué no hay máximo)
3. El struct Estrategia (la "ficha")
4. El puntero a función FuncionDecision
5. Todos los arreglos, dibujados
6. La regla de los índices: $N - 1$ y $N - 2$
7. Las 8 estrategias por dentro
8. Una partida, ronda por ronda
9. La matriz de pagos y el paso por referencia
10. El torneo: por qué $j = i + 1$
11. Estadísticas: victorias y porcentajes
12. El ranking: burbuja sobre índices
13. Lectura validada y salida por pantalla
14. Glosario de variables A–Z
15. Respuestas de los "Pruébalo"

1. El programa en 30 segundos

Todo empieza en `main_torneo.cpp`. La función `main()` no tiene lógica del juego: es un **director** que llama a otras funciones en orden. Cada número te lleva a la sección que explica ese paso.

```
int main()
{
    srand(time(0)); // semilla del azar 7

    Estrategia estrategias[NUM_ESTRATEGIAS]; // Los 8 jugadores 5
    int ranking[NUM_ESTRATEGIAS]; // orden final 12

    int numeroRondas = leerNumeroRondas(); // pedir rondas 13

    inicializarEstrategias(estragias); // Llenar las fichas 5
    ejecutarTorneo(estragias, numeroRondas); // 28 partidas 10
    calcularEstadisticas(estragias); // G / P / E y % 11
    ordenarRanking(estragias, ranking); // de mayor a menor 12

    mostrarDetalleTorneo(estragias, ranking); // imprimir 13
    mostrarRanking(estragias, ranking, numeroRondas);
    mostrarGanador(estragias, ranking);
    return 0;
}
```

El viaje de los datos

Piensa en el programa como una cadena de montaje. Cada función recibe el arreglo `estrategias`, le escribe algo y se lo pasa a la siguiente:

Paso	Función	¿Qué campos de cada <u>Estrategia</u> llena?
1	<code>inicializarEstrategias</code>	<code>id</code> , <code>nombre</code> , <code>decidir</code> ; todo lo demás en 0
2	<code>ejecutarTorneo</code> → <code>ejecutarPartida</code>	<code>puntosContra[]</code> , <code>puntajeTotal</code> , <code>vecesC</code> , <code>vecesT</code>
3	<code>calcularEstadisticas</code>	<code>victorias</code> , <code>derrotas</code> , <code>empates</code> , <code>porcentajeC</code> , <code>porcentajeT</code>
4	<code>ordenarRanking</code>	Ninguno: llena el arreglo aparte <code>ranking</code>
5	<code>mostrar...</code>	Ninguno: solo lee (por eso reciben <code>const</code>)

¿Por qué está dividido en tantos archivos? Cada archivo `.h/.cpp` tiene una sola responsabilidad. El `.h` es la “promesa” (qué funciones existen y qué reciben) y el `.cpp` es el “cómo”. Así, si quieres cambiar la matriz de pagos, solo tocas `matriz_pagos.cpp` y nada más se rompe.

Mapa de archivos

Archivo	Qué contiene	Sección
tipos_torneo.h	Constantes, FuncionDecision, struct Estrategia	2 , 3 , 4
estrategias_juego.cpp	Las 8 funciones decidirXxx e inicializarEstrategias	7 , 5
partida_iterada.cpp	ejecutarPartida, registrarJugada	8
matriz_pagos.cpp	calcularPuntosRonda	9
torneo_round_robin.cpp	ejecutarTorneo	10
estadisticas_torneo.cpp	calcularEstadisticas, ordenarRanking	11 , 12
lectura_rondas.cpp, reporte_resultados.cpp	Entrada por teclado y salida por pantalla	13

2. Las constantes: por qué 8 y 201 (y por qué no hay máximo)

```
const int NUM_ESTRATEGIAS = 8;
const int MIN Rondas = 201; // no hay MAX Rondas
```

Constante	Valor	¿Dónde se usa?	¿Por qué ese valor?
NUM_ESTRATEGIAS	8	Tamaño de estrategias , ranking , puntosContra y límite de todos los <code>for</code> del torneo.	6 estrategias obligatorias + 2 propias.
MIN_RONDAS	201	leerNumeroRondas	El enunciado exige más de 200 rondas → el menor entero válido es 201.

¿Por qué constantes y no escribir 8 directamente? Si mañana agregas una novena estrategia, cambias **un solo número** y todos los arreglos y bucles se ajustan solos. Con el 8 escrito en 15 lugares tendrías que encontrarlos todos (y es fácil olvidar uno).

¿Por qué no hay máximo? El enunciado solo pide **más de 200** rondas; no pone un límite superior, así que el programa tampoco. Eso obliga a tres decisiones:

- **Historial con memoria dinámica.** `char historialA[numeroRondas]` no es C++ estándar, porque el tamaño se conoce recién cuando el usuario escribe. Por eso `ejecutarPartida` crea los historiales con `new char[numeroRondas]` (el tamaño exacto que se pidió) y los libera con `delete[]` al terminar la partida ([sección 8](#)).
- **Puntajes long long.** Un `int` llega hasta 2 147 483 647. Con muchas rondas los puntajes pasarían ese valor y se desbordarían ([Pruébalo 1](#)). `long long` llega hasta unos $9,2 \times 10^{18}$.
- **Vengativa sin recorrer todo el historial.** Si en cada ronda se revisaran todas las rondas anteriores, con muchas rondas el torneo sería muy lento ([sección 7](#)).

El único tope práctico es que `numeroRondas` es un `int`: un número mayor que 2 147 483 647 no cabe y `cin` lo trata como entrada inválida. Además, mientras más rondas, más memoria y más tiempo tarda el torneo.

Pruébalo 1. Si el usuario escribe 100 000 000 rondas, ¿cuántos puntos puede llegar a tener, como máximo, una estrategia? ¿Cabrá en un `int`? → [respuesta](#)

Las constantes de la matriz de pagos

```
const int PUNTOS_COOPERACION_MUTUA = 3; // C-C
const int PUNTOS_TRAICION_EXITOSA = 5; // yo T, él C
const int PUNTOS_ENGANADO = 0; // yo C, él T
const int PUNTOS_TRAICION_MUTUA = 1; // T-T
```

Están dentro de `matriz_pagos.cpp` (no en el `.h`) porque solo esa función las necesita. Tener nombres evita los “números mágicos”: `puntosA = PUNTOS_ENGANADO` se entiende; `puntosA = 0` no dice por qué.

3. El struct Estrategia (la “ficha” de cada jugador)

Un `struct` agrupa variables que pertenecen a lo mismo. En vez de tener 12 arreglos sueltos (`nombres[8]`, `puntajes[8]`, `vecesC[8]` ...), cada estrategia tiene su propia ficha con todos sus datos juntos.

`struct Estrategia`

IDENTIDAD (NO CAMBIA)

```
int id
string nombre
FuncionDecision decidir
```

SE LLENA AL JUGAR

```
long long puntajeTotal
long long puntosContra[8]
long long vecesC
long long vecesT
```

SE CALCULA AL FINAL

```
int victorias
int derrotas
int empates
double porcentajeC
double porcentajeT
```

Campo	Para qué existe	Lo escribe	Lo lee
<code>id</code>	Saber qué posición ocupa en el arreglo.	<code>inicializarEstrategias</code>	—
<code>nombre</code>	Mostrarlo por pantalla.	<code>inicializarEstrategias</code>	<code>funciones mostrar...</code>
<code>decidir</code>	Guardar qué función usa esta estrategia para elegir C o T.	<code>inicializarEstrategias</code>	<code>ejecutarPartida</code>
<code>puntosContra[j]</code>	Puntos que sacó contra la estrategia <code>j</code> . Hace falta para el detalle y para saber si ganó o perdió cada partida.	<code>ejecutarTorneo</code>	<code>calcularEstadisticas</code> , <code>mostrarDetalle...</code>
<code>puntajeTotal</code>	Suma de todos sus <code>puntosContra</code> . Decide el ranking.	<code>ejecutarTorneo</code>	<code>ordenarRanking</code> , <code>mostrar...</code>
<code>vecesC</code> , <code>vecesT</code>	Cuántas veces cooperó / traicionó en todo el torneo.	<code>registrarJugada</code>	<code>calcularEstadisticas</code>
<code>victorias</code> , <code>derrotas</code> , <code>empates</code>	Resultado de sus 7 partidas.	<code>calcularEstadisticas</code>	<code>mostrar...</code>
<code>porcentajeC</code> , <code>porcentajeT</code>	Porcentajes (con decimales, por eso <code>double</code>).	<code>calcularEstadisticas</code>	<code>mostrar...</code>

¿Por qué `puntosContra` es un arreglo dentro del struct? Porque cada estrategia juega contra 8 posiciones posibles (7 rivales + ella misma, que queda en 0). La posición del arreglo es el `id` del rival: `estrategias[3].puntosContra[1]` = “lo que Tit for Tat (3) sacó contra Siempre Traicionar (1)”.

Pruébalo 2. ¿Qué valor tiene siempre `estrategias[5].puntosContra[5]` al final del torneo, y por qué? ➡ [respuesta](#)

¿Por qué todas las funciones reciben los 3 parámetros aunque algunas no los usen? Para caber en el mismo tipo de puntero, todas deben tener **exactamente la misma firma**. `decidirSiempreCooperar` ignora el historial, pero igual debe aceptarlo.

Pruébalo 3. Quieres agregar una novena estrategia, "Rencorosa suave". ¿Qué tres cosas tienes que cambiar en el código? → [respuesta](#)

5. Todos los arreglos del programa, dibujados

Arreglo	Tipo y tamaño	Dónde vive	Qué guarda
estrategias	Estrategia[8]	main	Las 8 fichas. Todo el programa trabaja sobre él.
ranking	int[8]	main	Índices ordenados por puntaje.
puntosContra	long long[8] dentro de cada ficha	struct	Puntos contra cada rival. Juntando los 8 forma una tabla 8 × 8.
historialA / historialB	new char[numeroRondas]	ejecutarPartida	Jugadas C/T de la partida actual. Tamaño exacto, se libera con <code>delete[]</code> .
nombres, funciones	string[8], FuncionDecision[8]	inicializarEstrategias	Datos para llenar las fichas (ver sección 4).

estrategias[8]: el arreglo de fichas

Cada celda es un struct completo. Al final del torneo (250 rondas) se ve así:

índice	nombre	puntajeTotal	vecesC	vecesT	G/P/E
0	Siempre Cooperar	3408	1750	0	0/3/4
1	Siempre Traicionar	3750	0	1750	7/0/0
2	Aleatoria	3734	876	874	3/2/2
3	Tit for Tat	4553	1372	378	0/1/6
4	Vengativa	4261	1004	746	2/1/4
5	Ojo por Dos Ojos	3950	1189	561	0/3/4
6	Pavlov	4184	1345	405	1/2/4
7	Sonda	3828	468	1282	2/3/2

¿Por qué 1750? Cada estrategia juega 7 partidas de 250 rondas: $7 \times 250 = 1750$ jugadas. Por eso `vecesC` + `vecesT` siempre suma 1750.

puntosContra: una tabla 8 × 8 repartida en 8 fichas

Si pones los 8 arreglos `puntosContra` uno debajo del otro, obtienes una tabla. La fila es "yo" y la columna es "mi rival":

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
0 S.Coop	0	0	402	750	750	750	750	6
1 S.Trai	1250	0	726	254	254	258	750	258
3 TFT	750	249	555	0	750	750	750	749
...	(filas 2, 4, 5, 6, 7)							

La celda resaltada es `estrategias[3].puntosContra[1] = 249`. Su "espejo" es `estrategias[1].puntosContra[3] = 254`. Comparando las dos se sabe quién ganó esa partida ([sección 11](#)).

ranking[8]: índices, no fichas

ranking[] después de ordenar

3	4	6	5	7	1	2	0
0	1	2	3	4	5	6	7

ranking[0] = 3 significa “el primer puesto lo tiene la estrategia 3 (Tit for Tat)”. Para leer su nombre:

estrategias[ranking[0]].nombre. [¿Por qué índices y no fichas? → sección 12](#)

6. La regla de los índices: $N - 1$ y $N - 2$

Las rondas se cuentan desde **1** (como las cuenta una persona), pero los arreglos en C++ empiezan en **0**. De ahí salen dos reglas que aparecen en todo el código:

Quiero...	Escribo...	Ejemplo en la ronda 5
Guardar la jugada de la ronda actual	<code>historial[ronda - 1]</code>	se guarda en la posición 4
Leer la jugada de la ronda anterior	<code>historial[numeroRonda - 2]</code>	se lee la posición 3 (= ronda 4)
Leer la de hace dos rondas	<code>historial[numeroRonda - 3]</code>	se lee la posición 2 (= ronda 3)

Así se ve `historialOponente` cuando se está decidiendo la **ronda 5** (4 rondas ya jugadas):

C	T	C	T	?	?	...	?
[0] r1	[1] r2	[2] r3	[3] r4	[4] r5	[5]	...	[numeroRondas - 1]

La celda con borde amarillo es `historialOponente[5 - 2]`: la última jugada del rival. Las celdas con **?** tienen basura (el arreglo no se inicializa), pero **nunca se leen**: cada estrategia solo mira posiciones menores que `numeroRonda - 1`.

¿Por qué Tit for Tat, Vengativa y Pavlov tienen `if (numeroRonda == 1)`? En la ronda 1, `numeroRonda - 2 = -1`. Leer `historial[-1]` sería leer memoria fuera del arreglo. Esa condición es una **protección**, no solo una regla del juego. Igual con Ojo por Dos Ojos, que necesita `numeroRonda >= 3` para leer `numeroRonda - 3`.

Pruébalo 4. En la ronda 10, ¿qué posiciones de los historiales lee `decidirVengativa`? → [respuesta](#)

7. Las 8 estrategias por dentro

Todas tienen la misma firma: reciben `numeroRonda`, `historialPropio` e `historialOponente`, y devuelven `'C'` o `'T'`.

Estrategia	Código clave	¿Qué mira?
Siempre Cooperar	<code>return 'C';</code>	Nada.
Siempre Traicionar	<code>return 'T';</code>	Nada.
Aleatoria	<code>if (rand() % 2 == 0) return 'C'; return 'T';</code>	Nada: el azar.
Tit for Tat	<code>return historialOponente[numeroRonda - 2];</code>	Última jugada del rival.
Vengativa	<code>if (historialOponente[numeroRonda - 2] == 'T' historialPropio[numeroRonda - 2] == 'T') return 'T';</code>	La última del rival y la última propia.
Ojo por Dos Ojos	<code>if (anterior == 'T' && antepasada == 'T') return 'T';</code>	Las dos últimas del rival.
Pavlov	<code>if (miJugadaAnterior == jugadaAnteriorRival) return 'C';</code>	La última propia y la última del rival.
Sonda	<code>oponenteSeDefendio = (historialOponente[1]=='T' historialOponente[2]=='T');</code>	Rondas 2 y 3 del rival + la última.

¿Por qué `srand(time(0))` está en `main` y no en `decidirAleatoria`? `srand` fija el punto de partida del azar. Si se llamara en cada ronda, todas las rondas del mismo segundo tendrían la misma semilla y darían **la misma jugada**. Se llama una sola vez al inicio y después `rand()` va dando números distintos.

Vengativa: “recordar para siempre” mirando solo la ronda anterior

```
if (numeroRonda == 1) return 'C';
if (historialOponente[numeroRonda - 2] == 'T' || historialPropio[numeroRonda - 2] == 'T')
    return 'T';
return 'C';
```

No tiene una variable `bool` `yaMeTraicionaron`. Hay solo dos formas de que Vengativa traicione en esta ronda:

- **El rival traicionó en la ronda anterior** → la venganza empieza ahora.
- **Yo traicioné en la ronda anterior** → eso solo pasa si la venganza ya había empezado, así que sigo.

La primera **T** del rival “enciende” la venganza, y desde ahí su propia **T** la mantiene encendida para siempre. La memoria **es** su propio `historial`.

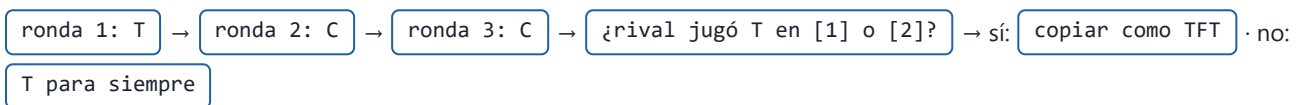
¿Por qué no recorrer todo el historial del rival buscando una **T?** Daría el mismo resultado, pero en la ronda n habría que revisar $n - 1$ posiciones. En una partida eso suma $1 + 2 + \dots + n \approx n^2 / 2$ lecturas: con 100 000 rondas serían unos 5 000 millones por partida. Como ya no hay límite de rondas, se mira solo la ronda anterior: **2 lecturas por ronda**, sin importar cuántas rondas se jueguen.

Pavlov: 4 casos en 1 regla

Yo (anterior)	Rival (anterior)	¿Iguales?	Jugada
C	C	sí	C
T	T	sí	C
C	T	no	T
T	C	no	T

Variables locales: `miJugadaAnterior = historialPropio[numeroRonda - 2]` y `jugadaAnteriorRival = historialOponente[numeroRonda - 2]`. Pavlov y Vengativa son las únicas estrategias que usan `historialPropio`: las demás solo miran al rival.

Sonda: la variable `oponenteSeDefendio`



¿Por qué no usar una variable `static bool` para recordar el modo? Una variable `static` dentro de una función **conserva su valor entre llamadas... y entre partidas**. La Sonda recordaría lo que decidió contra el rival anterior y lo aplicaría al siguiente. Calcularla desde el historial garantiza que cada partida empiece limpia, porque el historial se crea de nuevo en cada partida.

Pruébalo 5. Sonda contra Siempre Cooperar: ¿qué juega la Sonda en la ronda 4 y por qué? → [respuesta](#)

8. Una partida, ronda por ronda (ejecutarPartida)

```
void ejecutarPartida(Estrategia &estrategiaA, Estrategia &estrategiaB, int numeroRondas,
                    long long &puntosPartidaA, long long &puntosPartidaB)
{
    char *historialA = new char[numeroRondas];           1
    char *historialB = new char[numeroRondas];
    puntosPartidaA = 0; puntosPartidaB = 0;

    for (int ronda = 1; ronda <= numeroRondas; ronda++)
    {
        char jugadaA = estrategiaA.decidir(ronda, historialA, historialB);  2
        char jugadaB = estrategiaB.decidir(ronda, historialB, historialA);  3

        historialA[ronda - 1] = jugadaA;                                   2
        historialB[ronda - 1] = jugadaB;

        registrarJugada(estrategiaA, jugadaA);                             4
        registrarJugada(estrategiaB, jugadaB);

        int puntosRondaA; int puntosRondaB;
        calcularPuntosRonda(jugadaA, jugadaB, puntosRondaA, puntosRondaB);  9
        puntosPartidaA += puntosRondaA;
        puntosPartidaB += puntosRondaB;
    }

    delete[] historialA;                                                    1
    delete[] historialB;
}
```

1. **Historiales nuevos en cada partida:** se crean dentro de la función, así que ninguna estrategia “recuerda” lo que pasó contra otro rival. Como no hay máximo de rondas, se piden con `new char[numeroRondas]`: justo el tamaño necesario, ni una celda de más. Todo lo que se pide con `new` hay que devolverlo con `delete[]`; si no, cada una de las 28 partidas dejaría memoria ocupada sin usar (una **fuga de memoria**).
2. **Decidir primero, guardar después:** las dos estrategias deciden **antes** de guardar. Si se guardara `jugadaA` antes de que B decida, B podría ver lo que A acaba de jugar, y eso sería hacer trampa: en el dilema las decisiones son **simultáneas**.
3. **Los historiales se pasan al revés para B:** `(ronda, historialB, historialA)`. Para B, “propio” es el historial B y “oponente” es el A. Gracias a eso la misma función sirve para jugar de un lado o del otro.
4. **registrarJugada** suma 1 a `vecesC` o a `vecesT`. Recibe la estrategia **por referencia** (`&`) para modificar la ficha original y no una copia.

Traza: Tit for Tat (A) contra Sonda (B)

Ronda	¿Qué pasa en la cabeza de cada una?	A	B	+A	+B	Total A	Total B
1	TFT: ronda 1 → C. Sonda: ronda 1 → T.	C	T	0	5	0	5
2	TFT copia historialB[0] = T. Sonda: ronda 2 → C.	T	C	5	0	5	5
3	TFT copia historialB[1] = C. Sonda: ronda 3 → C.	C	C	3	3	8	8
4	Sonda: historialA[1] = T → se defendió → copia historialA[2] = C.	C	C	3	3	11	11
5...250	Ambas copian C: cooperación para siempre.	C	C	3	3	...	...
Final (250 rondas): $5 + 0 + 248 \times 3 =$						749	749

Así quedan los historiales después de la ronda 4:

historialA (Tit for Tat)

C	T	C	C	?
0	1	2	3	4

historialB (Sonda)

T	C	C	C	?
0	1	2	3	4

Traza: Pavlov contra Siempre Traicionar (su punto débil)

Ronda	Pavlov mira (yo, rival)	Pavlov	S. Trai.	+P	+ST
1	ronda 1	C	T	0	5
2	(C, T) distintas	T	T	1	1
3	(T, T) iguales	C	T	0	5
4	(C, T) distintas	T	T	1	1
...	se repite C, T, C, T... → $125 \times 0 + 125 \times 1 = 125$ contra 750				

Pruébalo 6. ¿Qué pasaría si en `ejecutarPartida` se escribiera `historialA[ronda - 1] = jugadaA`; justo después de calcular `jugadaA` (antes de `jugadaB`), en la partida Tit for Tat (A) contra Tit for Tat (B)? → [respuesta](#)

9. La matriz de pagos y el paso por referencia

```
void calcularPuntosRonda(char jugadaA, char jugadaB, int &puntosA, int &puntosB)
{
    if      (jugadaA == 'C' && jugadaB == 'C') { puntosA = 3; puntosB = 3; }
    else if (jugadaA == 'C' && jugadaB == 'T') { puntosA = 0; puntosB = 5; }
    else if (jugadaA == 'T' && jugadaB == 'C') { puntosA = 5; puntosB = 0; }
    else                                     { puntosA = 1; puntosB = 1; }
} // (en el código real se usan las constantes PUNTOS_...)
```

¿Por qué `int &puntosA con &`? Una función solo puede devolver **un** valor con `return`, pero aquí necesitamos devolver **dos** (los puntos de A y los de B). Con `&`, `puntosA` no es una copia: es **otro nombre** para la variable `puntosRondaA` de `ejecutarPartida`. Lo que se asigna dentro aparece afuera.

Parámetro	Cómo se pasa	¿Por qué?
<code>char jugadaA</code>	por valor (copia)	Solo se lee; copiar un <code>char</code> es gratis.
<code>int &puntosA</code>	por referencia	Es un resultado: la función lo escribe.
<code>Estrategia &estrategiaA</code>	por referencia	<code>ejecutarPartida</code> modifica <code>vecesC / vecesT</code> de la ficha original.
<code>const Estrategia estrategias[]</code>	arreglo, de solo lectura	Las funciones <code>mostrar...</code> y <code>ordenarRanking</code> no deben modificar nada; <code>const</code> hace que el compilador lo impida.
<code>Estrategia estrategias[]</code>	arreglo (siempre se comparte)	En C++ un arreglo nunca se copia al pasarlo: la función trabaja sobre el original. Por eso <code>ejecutarTorneo</code> puede llenar los puntajes.

Pruébalo 7. Si quitamos el `&` de `int &puntosA`, el programa compila. ¿Qué puntaje tendría cada estrategia al final? → [respuesta](#)

10. El torneo: por qué `i + 1`

```
for (int i = 0; i < NUM ESTRATEGIAS; i++)
    for (int j = i + 1; j < NUM ESTRATEGIAS; j++)
    {
        long long puntosI; long long puntosJ;
        ejecutarPartida(estrategias[i], estrategias[j], numeroRondas, puntosI, puntosJ);

        estrategias[i].puntosContra[j] = puntosI;    // desde el punto de vista de i
        estrategias[j].puntosContra[i] = puntosJ;    // desde el punto de vista de j
        estrategias[i].puntajeTotal += puntosI;
        estrategias[j].puntajeTotal += puntosJ;
    }
```

Cada número de la tabla es el **orden** en que se juega esa partida. Solo se juega el triángulo superior:

i \ j	0	1	2	3	4	5	6	7
0		1	2	3	4	5	6	7
1	=1		8	9	10	11	12	13
2	=2	=8		14	15	16	17	18
3	=3	=9	=14		19	20	21	22
4	=4	=10	=15	=19		23	24	25
5	=5	=11	=16	=20	=23		26	27
6	=6	=12	=17	=21	=24	=26		28
7	=7	=13	=18	=22	=25	=27	=28	

¿Por qué `j` empieza en `i + 1` y no en 0?

- Si empezara en 0, se jugaría 0 contra 1 **y también** 1 contra 0: cada partida dos veces (56 en vez de 28) y todos los puntajes se duplicarían.
- Si empezara en `i`, cada estrategia jugaría contra sí misma (la diagonal gris).
- Con `i + 1` se juega exactamente cada par una vez: $7 + 6 + 5 + 4 + 3 + 2 + 1 = 28$ **partidas** = $8 \times 7 / 2$.

¿Por qué se guardan dos celdas por partida? La partida 9 (1 contra 3) llena

`estrategias[1].puntosContra[3]` y también `estrategias[3].puntosContra[1]`. Así el triángulo inferior (las celdas "=9") también queda lleno sin jugar de nuevo, y cada estrategia tiene sus 7 resultados.

Pruébalo 8. ¿Cuántas partidas se jugarían con 10 estrategias? ¿Y cuántas partidas juega cada una? → [respuesta](#)

11. Estadísticas: victorias y porcentajes

```
for (int j = 0; j < NUM ESTRATEGIAS; j++)
{
    if (j == i) continue; // no hubo partida contra sí misma
    long long misPuntos = estrategias[i].puntosContra[j];
    long long puntosRival = estrategias[j].puntosContra[i]; // la celda "espejo"
    if (misPuntos > puntosRival) estrategias[i].victorias++;
    else if (misPuntos < puntosRival) estrategias[i].derrotas++;
    else
        estrategias[i].empates++;
}
```

Ejemplo con $i = 3$ (Tit for Tat) y $j = 1$ (Siempre Traicionar): `misPuntos = 249`, `puntosRival = 254` → `249 < 254` → `derrotas++`.

¿Por qué **continue** cuando $j == i$? `puntosContra[i]` de sí misma vale 0, y 0 contra 0 contaría como un **empate falso**. Con `continue` se salta esa vuelta del bucle y cada estrategia suma exactamente 7 resultados.

Porcentajes

```
long long totalJugadas = estrategias[i].vecesC + estrategias[i].vecesT;
if (totalJugadas > 0)
    estrategias[i].porcentajeC = 100.0 * estrategias[i].vecesC / totalJugadas;
```

¿Por qué **100.0** y no **100**? Cuando todos los operandos son enteros (`int` o `long long`), C++ hace **división entera** y descarta los decimales. Con `100 * 1372 / 1750` saldría 78 en vez de 78.40; y si alguien escribiera `vecesC / totalJugadas * 100`, saldría 0, porque `1372 / 1750 = 0` en división entera. Al empezar con `100.0` (un `double`), toda la operación se hace con decimales: `100.0 * 1372 / 1750 = 78.40`.

¿Por qué el **if (totalJugadas > 0)**? Para no dividir entre cero. En este programa nunca pasa (siempre hay jugadas), pero es una protección barata.

Pruébalo 9. Siempre Traicionar tiene 7 victorias y queda 6.^a. Tit for Tat tiene 0 victorias y queda 1.^a. ¿Cómo es posible? → [respuesta](#)

12. El ranking: burbuja sobre índices

```
for (int i = 0; i < NUM_ESTRATEGIAS; i++) ranking[i] = i;           // 0,1,2,...,7

for (int pasada = 0; pasada < NUM_ESTRATEGIAS - 1; pasada++)
    for (int k = 0; k < NUM_ESTRATEGIAS - 1 - pasada; k++)
    {
        long long puntajeActual    = estrategias[ranking[k]].puntajeTotal;
        long long puntajeSiguiente = estrategias[ranking[k + 1]].puntajeTotal;
        if (puntajeActual < puntajeSiguiente)           // el de abajo tiene más: intercambiar
        {
            int temporal = ranking[k]; ranking[k] = ranking[k + 1]; ranking[k + 1] = temporal;
        }
    }
}
```

¿Por qué ordenar índices y no las fichas? Si se intercambiaran las fichas, `estrategias[3]` dejaría de ser Tit for Tat. Pero `puntosContra[3]` de todas las demás sigue significando "contra la estrategia que estaba en la posición 3". Toda la tabla de resultados quedaría desordenada. Ordenando un arreglo aparte de índices, `estrategias[]` no se mueve. Además, intercambiar dos `int` es más barato que intercambiar dos structs con un `string` y un arreglo dentro.

¿Por qué `k < NUM_ESTRATEGIAS - 1 - pasada`? Después de cada pasada, el menor puntaje "se hunde" hasta el final y queda en su lugar definitivo. No hace falta volver a compararlo. Y el `- 1` evita que `k + 1` se salga del arreglo.

Traza con los puntajes reales

Puntajes por índice: 0→3408, 1→3750, 2→3734, 3→4553, 4→4261, 5→3950, 6→4184, 7→3828.

Momento	ranking[0 ... 7]								intercambios
inicio	0	1	2	3	4	5	6	7	—
tras pasada 0	1	2	3	4	5	6	7	0	7
tras pasada 1	1	3	4	5	6	7	2	0	5
tras pasada 2	3	4	5	6	7	1	2	0	5
tras pasada 3	3	4	6	5	7	1	2	0	1
pasadas 4-6	3	4	6	5	7	1	2	0	0

En la pasada 0, Siempre Cooperar (índice 0, el menor puntaje) se intercambia 7 veces hasta llegar al final. Resultado: TFT, Vengativa, Pavlov, Ojo por Dos Ojos, Sonda, S. Traicionar, Aleatoria, S. Cooperar.

Pruébalo 10. ¿Qué habría que cambiar para ordenar de **menor** a mayor? → [respuesta](#)

13. Lectura validada y salida por pantalla

leerNumeroRondas()

```
int numeroRondas = 0;
bool entradaValida = false;
while (!entradaValida)
{
    cout << "Ingrese el numero de rondas por partida (mayor a 200): ";
    cin >> numeroRondas;
    if (cin.fail()) {
        // escribió letras
        cin.clear(); // 1. quita el estado de error
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // 2. bota la línea
        cout << "Entrada invalida...";
    } else {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        if (numeroRondas < MIN Rondas) cout << "El numero de rondas debe ser mayor a 200.\n";
        else entradaValida = true;
    }
}
```

Variable	¿Por qué existe?
<code>entradaValida</code>	Una bandera (<code>bool</code>) que controla el <code>while</code> : repetir hasta que la entrada sirva.
<code>cin.clear()</code>	Cuando <code>cin</code> falla, queda "bloqueado" y todas las lecturas siguientes fallan. <code>clear()</code> lo desbloquea.
<code>cin.ignore(...)</code>	Las letras que causaron el error siguen esperando en la entrada. Si no se botan, el <code>cin >></code> siguiente las vuelve a leer y el programa entra en un bucle infinito.
<code>numeroRondas < MIN Rondas</code>	Es la única comparación: solo se controla el mínimo, porque el enunciado no pone un máximo. Un número que no cabe en un <code>int</code> (más de 2 147 483 647) hace fallar a <code>cin</code> y cae en "Entrada invalida".

Funciones de salida

- `mostrarDetalleTorneo` recorre `ranking[]` y llama a `mostrarDetalleEstrategia(estrategias, ranking[puesto])` : el detalle sale **en orden de puesto**.
- `setw(n)` reserva `n` espacios para el siguiente dato y `left / right` lo alinean: así la tabla sale en columnas.
- `fixed << setprecision(2)` muestra los `double` con dos decimales (78.40).
- `const Estrategia &estrategia = estrategias[indice];` crea un **alias corto** para no escribir `estrategias[indice]`. en cada línea. Es `const &`: no copia y no permite modificar.
- `mostrarGanador` compara con `puntajeMaximo` para anunciar a **todas** las estrategias empatadas en el primer lugar, no solo a `ranking[0]`.

14. Glosario de variables A–Z

Nombre	Tipo	Archivo / función	Qué es y por qué existe
decidir	FuncionDecision	struct Estrategia	Dirección de la función que elige C/T. → 4
derrotas / empates	int	struct Estrategia	Partidas perdidas / empatadas. → 11
entradaValida	bool	leerNumeroRondas	Bandera que detiene el while de validación. → 13
estrategias	Estrategia[8]	main	Las 8 fichas de los jugadores. → 5
estrategiaA / estrategiaB	Estrategia&	ejecutarPartida	Las dos fichas que juegan (por referencia para actualizar sus contadores). → 8
funciones	FuncionDecision[8]	inicializarEstrategias	Arreglo paralelo a nombres con las 8 funciones. → 4
historialA / historialB	char* (new char[numeroRondas])	ejecutarPartida	Jugadas de cada lado en la partida actual; se liberan con delete[] . Se pasan como historialPropio / historialOponente según quién decide. → 6
historialPropio / historialOponente	const char[]	funciones decidir...	Lo que la estrategia "ve". const : una estrategia no puede cambiar el pasado. → 7
id	int	struct Estrategia	Posición de la ficha en estrategias[] .
i, j	int	ejecutarTorneo, calcularEstadisticas	i = estrategia actual, j = rival. → 10
jugadaA / jugadaB	char	ejecutarPartida	Decisión de cada una en la ronda actual. → 8
jugadaAnterior / jugadaAntepasada	char	decidirOjoPorDosOjos	Las dos últimas jugadas del rival.
k, pasada, temporal	int	ordenarRanking	Contadores de la burbuja y variable auxiliar del intercambio. → 12

miJugadaAnterior / jugadaAnteriorRival	char	decidirPavlov	Lo que jugaron ambos en la ronda anterior. → 7
misPuntos / puntosRival	long long	calcularEstadisticas	Las dos celdas espejo de una partida. → 11
nombre / nombres	string / string[8]	struct / inicializar...	Texto que se imprime.
numeroRonda	int	funciones decidir...	Ronda que se está decidiendo (empieza en 1). → 6
numeroRondas	int	main	Rondas por partida que elige el usuario (mayor a 200, sin máximo). → 13
oponenteSeDefendio	bool	decidirSonda	¿El rival respondió a la traición inicial? → 7
porcentajeC / porcentajeT	double	struct Estrategia	% de C y T en todo el torneo. → 11
puntajeTotal	long long	struct Estrategia	Suma de sus 7 partidas; decide el ranking. → 10
puntajeActual / puntajeSiguiente	long long	ordenarRanking	Los dos puntajes que se comparan en cada paso.
puntosContra	long long[8]	struct Estrategia	Puntos contra cada rival. → 5
puntosI / puntosJ	long long	ejecutarTorneo	Resultado de la partida actual (los llena ejecutarPartida por referencia).
puntosPartidaA / B	long long&	ejecutarPartida	Acumulan los puntos de toda la partida.
puntosRondaA / B	int	ejecutarPartida	Puntos de una sola ronda. → 9
puntajeMaximo	long long	mostrarGanador	Puntaje del primer lugar, para detectar empates.
ranking	int[8]	main	Índices ordenados de mayor a menor. → 5
ronda	int	ejecutarPartida	Contador del bucle de rondas (1 ... numeroRondas).
totalJugadas	long long	calcularEstadisticas	vecesC + vecesT , divisor de los porcentajes.
vecesC / vecesT	long long	struct Estrategia	Cuántas veces cooperó / traicionó. Los suma registrarJugada .

victorias	int	struct Estrategia	Partidas con más puntos que el rival. → 11
-----------	-----	-------------------	--

15. Respuestas de los “Pruébalo”

- 1 **No cabría.** Lo máximo por ronda son 5 puntos y cada estrategia juega 7 partidas: $7 \times 5 \times 100\,000\,000 = 3\,500\,000\,000$, más que el máximo de un `int` (2 147 483 647). Con `int` el puntaje se desbordaría y saldría un número absurdo, incluso negativo, y el ranking quedaría mal. Por eso `puntajeTotal`, `puntosContra`, `vecesC` y `vecesT` son `long long`. Los historiales no tienen problema: se crean con `new` del tamaño exacto. [← volver](#)
- 2 **Vale 0.** Lo pone en 0 `inicializarEstrategias` y nunca se modifica, porque `j = i + 1` impide que una estrategia juegue contra sí misma. [← volver](#)
- 3 (1) Escribir la función `char decidirRencorosaSuave(int, const char[], const char[])` y declararla en `estrategias_juego.h`. (2) Agregar su nombre al arreglo `nombres` y su función al arreglo `funciones`. (3) Cambiar `NUM_ESTRATEGIAS` a 9. No hace falta tocar `ejecutarPartida` ni `ejecutarTorneo`: esa es la ventaja del puntero a función. [← volver](#)
- 4 Solo la posición `10 - 2 = 8` (la ronda 9), en los dos historiales: `historialOponente[8]` y `historialPropio[8]`. No necesita las rondas 1 a 8: si el rival traicionó en alguna, Vengativa ya habría jugado **T** en la ronda 9 y `historialPropio[8]` lo mostraría. [← volver](#)
- 5 **T.** Siempre Cooperar jugó C en las rondas 2 y 3 (`historialOponente[1]` y `[2]`), así que `oponenteSeDefendio = false` y la Sonda entra en modo explotar. Resultado final: $5 + 3 + 3 + 247 \times 5 = 1246$ contra 6. [← volver](#)
- 6 En este caso el resultado **no cambia** (750 – 750): cuando B decide la ronda `r`, Tit for Tat lee `historialA[r - 2]`, la ronda anterior, y nunca la posición `r - 1` que A acaba de escribir. Pero el programa quedaría frágil: si alguien agrega una estrategia que lea `historialOponente[numeroRonda - 1]`, B podría ver la jugada actual de A antes de decidir (A no podría hacer lo mismo con B). El orden “primero deciden las dos, luego se guarda” garantiza que las decisiones sean simultáneas y que nadie pueda hacer trampa. [← volver](#)
- 7 Sin `&`, la función asignaría los puntos a una **copia local** que desaparece al terminar. `puntosRondaA` y `puntosRondaB` en `ejecutarPartida` nunca se inicializan (tienen basura), así que los puntajes serían números basura o cero, según el compilador. [← volver](#)
- 8 $10 \times 9 / 2 = 45$ **partidas**, y cada estrategia juega **9** (contra todas menos ella misma). [← volver](#)
- 9 Ganar una partida solo significa sacar más que el rival, aunque ambos saquen poco. Siempre Traicionar gana 254 – 249 contra Tit for Tat: gana, pero solo con 254 puntos. Tit for Tat empató 750 – 750 contra las estrategias amables: no gana, pero suma 750. El ranking usa `puntajeTotal`, no `victorias`. [← volver](#)
- 10 Cambiar la comparación `if (puntajeActual < puntajeSiguiente)` por `if (puntajeActual > puntajeSiguiente)`. Nada más. [← volver](#)